

R.O.B. VISION

VIRTUAL ROBOT COMPANION / MK.01

TECHNICAL REFERENCE

Architecture, frame protocol, networking, hardware, and verification

CURRENT REFERENCE / 25 SEPTEMBER 2026

UNO Q + RETROPIE LIVE · FRAME LINK VERIFIED



CHAPTER 01 / CURRENT REFERENCE

R.O.B. Vision technical architecture

Status, 25 September 2026: The Arduino UNO Q runs the App Lab controller, matrix sketch, HTTP dashboard, and virtual R.O.B. model. RetroPie launch hooks and a paired virtual Controller 2 receiver run on `retropie.local`. FCEUmm delivered all six command types from both games during live Direct play; Nestopia delivered live commands from both games. Both Gyromite gate colors responded in Game A under FCEUmm. All robot parts are software state and graphics. No camera path remains.

Runtime path

```
RetroPie runcommand -- authenticated launch/exit --> UNO Q controller
NES frames --> FCEUmm or Nestopia wrapper --> local frame receiver
--> authenticated command/Test signal --> UNO Q virtual model
-- HTTP snapshot --> browser
-- Gyromite pads --> RetroPie Controller 2
```

The App Lab `python/main.py` gateway publishes port 80 and forwards to `controller/service.py` on port 8766. Router Bridge sends controller status and action hints to `sketch/sketch.ino` for the UNO Q LED matrix. The dashboard polls `/api/state` every 500 ms. A local file preview uses a separate scripted JavaScript model; a served page with a controller enters live mode.

Authority and state

`controller/service.py` owns the selected game, Test state, recent events, and virtual game models. `/api/state` schema 1 contains game, robot, input, events, link, and Test data. `input.frame_hook` reports a fresh RetroPie game-frame connection. Recent events are capped at 30 and are display history, not a durable event stream. Browser reload reads a new snapshot without resetting controller state.

`controller/model.py` keeps Gyromite's two gyros and Stack-Up's five blocks. Complete game-frame commands and manual commands pass through the same model. Invalid moves leave pieces in place. Stack-Up starts with all five blocks on Tray 3, and a grip may carry a block with those above it. Bingo's distinct starting layout is not implemented. Gyromite's 55-second spin lifetime and 60-second Fast Gate hold are illustrative settings.

The Gyromite pad reducer derives red and blue from gyro location/spin or a held unspun gyro. The RetroPie service polls the UNO Q, creates a Linux uinput controller, and applies states only while Gyromite is the active RetroArch game. It releases both buttons on exit, unknown game, missing process, failed authentication, or stale state. The verified port 2 mapping makes the dashboard's blue control move a blue gate and red control move a red gate under FCEUmm.

Game-frame decoding

`rob_vision_fceumm_proxy.c` builds one wrapper for FCEUmm and one for Nestopia. Each delegates libretro API calls to the installed original core. Its video callback classifies each rendered NES frame as dark, green, or other and sends the frame number and class through a local Unix datagram socket. `retropie_frame_hook.py` verifies the sender's process credentials, approved wrapper path, and exact ROM identity. `controller/optical.py` accepts only complete game-appropriate 13-frame patterns. The receiver forwards commands to token-protected `/api/emulator/command`; retries are idempotent by sender PID and frame number.

The same classified frames recognize Gyromite's sustained green Test field or Stack-Up's alternating Test frames. The receiver reports a recent Test signal in its authenticated heartbeat, and the UNO Q blinks R.O.B.'s red light automatically. It returns to the game display after the signal stops. A separate READY command lights it steadily for at most one second, so a ready command sent while exiting Test cannot leave the matrix in T mode. Neither Test indication moves the model. FCEUmm remains selected for both registered games on the test RetroPie. Nestopia's Gyromite Player 2 return passed Game A blue-gate checks on Batocera and RetroPie after the wrapper selected explicit gamepad device 257. See the [Nestopia report](#).

Batocera host

Batocera 43.1 x86_64 uses the same frame decoder, UNO Q model, and uinput receiver. Its separate R0BVision service installs packaged x86_64 wrappers for stock FCEUmm and Nestopia in a runtime core overlay. The `zz-robvision-game` hook reports exact Gyromite/Stack-Up launch and exit events. It coexists with VirtualGlove's service, hook, and Super Glove Ball core. Batocera generates `retroarchcustom.cfg` after game-start hooks, so the service detects its virtual pad at startup and writes Gyromite-only `retroarch.*` per-ROM overrides to persistent `batocera.conf`. The Linux `/dev/input/jsN` number is not the RetroArch joystick index: during live 0.1.0 validation, the virtual pad was `js4` but RetroArch enumerated it as pad 2. The receiver uses Batocera's SDL2 joystick order and writes that detected index with A button 1 and B button 0. Nestopia interprets device 1 as Auto and Gyromite selects its optical peripheral on Player 2; the wrapper now selects explicit gamepad device 257 after ROM load. Batocera's Game A blue-gate hold and release passed with that fix.

Both console receivers identify themselves in authenticated heartbeats, launch events, and decoded commands. The UNO Q accepts frame commands from the active game's console and rejects the other. The inactive receiver releases its virtual Gyromite pad state. If both consoles are online, the status shows the active game's source; while idle, it may show either online receiver.

HTTP and trust boundary

Trusted-LAN browser controls use `/api/game`, `/api/command`, and `/api/gate-assist` without a token. The browser reads `/api/state` and matrix status. JSON and same-origin checks protect browser writes. `/api/launch` and `/api/emulator/command` require the originating console's credential. Pairing uses a time-limited code and verified TLS certificate fingerprint to deliver it. Keep ports 80 and 8766 off untrusted networks.

Resets and limits

Selecting a game or **Home** initializes its virtual pieces. Live **Emergency Stop** clears game selection; file preview cancels its local animation. Game exit clears context and releases buttons. The RetroPie receiver resends game identity after a UNO Q restart. It does not infer Hector's position, gate animation, Stack-Up score, or victory.

See [configuration](#), [network architecture](#), [frame input](#), and the [verification plan](#).

0.1.0 release installation path

`scripts/package_release.py` packages a committed Git tree under `rob-vision/`, calculates its SHA-256 digest, and renders `scripts/release-install.sh` with the release tag and digest. The release also carries four rebuilt PDF guides and SHA256SUMS. The downloaded installer checks the archive digest before extraction or platform mutation, detects `uno-q` or `console`, and calls the existing platform installer. Console detection distinguishes Batocera's persistent configuration and core paths from standard RetroPie. Batocera checks its version file and CPU architecture before writes; version 0.1.0 has validated wrappers for Batocera 43.1 x86_64 only.

The RetroPie installer copies its own `scripts` directory into `/home/pi/rob-vision`, enables the receiver unit, and preserves the private credential on upgrade. On first install the bootstrap starts `retropie_pair.py` as root. A successful certificate-pinned pairing enables and restarts the receiver and invokes the installed `player2` detector to write the actual joystick index to the NES RetroArch config. Both console installers install the virtual pad's udev RetroArch profile, which a clean system may lack. Batocera's installer suspends its EmulationStation process before stopping and replacing receiver modules, restarts `ROBvision`, and resumes the menu in a `finally` block. Pairing also restarts `ROBvision`, whose service writes Gyromite-only Player 2 overrides before gameplay. Both paths retain unrelated games and VirtualGlove configuration. The console command's optional `--pair` repeats pairing after a credential has been revoked in Setup.

The release installer requires `curl`, `tar`, Python 3, and SHA-256 tooling on the device. The Uno Q installer keeps its App Lab stop, stage, start, and rollback behavior. The release does not ship ROMs or vendor emulator cores.

CHAPTER 02 / CURRENT REFERENCE

Original R.O.B. manual: historical notes

Source: [R.O.B. instruction manual transcript](#) at Atari HQ. This is a paraphrased reading of the original physical toy, not a hardware specification for R.O.B. Vision.

The original robot grasped/released, raised/lowered, and rotated/carried objects. Its operator aimed its eyes at a CRT test signal. A head light indicated optical reception and readiness, went out while it moved, and returned when ready for another command. R.O.B. Vision represents those movements and states in a browser; RetroPie passes rendered game frames to the UNO Q. There is no physical R.O.B. head, arm, or accessory.

The manual warns about glare, fluorescent light, blocked sight lines, overlays, and overly bright images. Those are historical concerns for the physical robot. The virtual UI separates **game frames linked** from **complete command decoded** and shows TEST → READY → BUSY → READY clearly. The original CRT distance is only historical context.

The original manual's battery, mechanical, and spinning-gyro safety instructions apply to Nintendo's toy. This virtual project instead needs robust command validation, correct virtual piece state, a fresh paired Gyromite return link, and clear simulation/live labels. See [optical input](#) and the [virtual system contract](#).

CHAPTER 03 / CURRENT REFERENCE

Gyromite manual: design and test notes

Source: [Gyromite instruction booklet transcript](#) at Digital Press. This is a paraphrased summary of Nintendo's physical game accessory, translated into rules for a virtual R.O.B. The transcript includes editorial notes and possible transcription mistakes; confirm game behavior with a user-supplied game before claiming compatibility.

Original accessory and virtual equivalent

The original set has one motorized spinner, two gyro holders, red/blue gyro trays, removable hands, and a holder for Controller 2. A gyro on the blue tray lowers the blue gate; one on red lowers the red gate. The original trays press Controller 2 mechanically. R.O.B. Vision draws these objects and derives a **virtual** red/blue button state in UNO Q software, then sends it over Wi-Fi or Ethernet to a paired virtual Controller 2 receiver. There are no physical gyros, trays, or contact sensors in this project. The transcript does not unambiguously map tray colors to Controller 2 A/B. On the RetroPie test setup, an isolated blue virtual pad press moved the blue Game A gate after its A/B mapping was corrected; the player later confirmed that both red and blue controls work.

The booklet says that **one gate can be operated without spinning a gyro**. R.O.B. may hold an unspun gyro down on a pad, then lift it to release. When both gates need control, a spinning gyro remains upright on one pad while R.O.B.'s hands move the other. The original setup begins with both gyros in holders, and the instructions describe R.O.B. moving a gyro between holder, spinner, and tray. The virtual controller therefore treats holders as resting positions and the spinner as an optional step driven by game needs.

Game modes

Mode	Manual behavior	Virtual design implication
Test	Optical aiming signal	Report Test signal from the rendered frame link before play.
Direct	Up/down/left/right/open/close go straight to R.O.B.	Validate six optical commands and show each virtual movement.
Game A	Player controls the professor on a dark screen; Start enters a blue robot-transmission screen	Decode complete robot commands in the correct mode; avoid claiming to know professor position from flashes.
Game B	Professor walks while Controller 1 commands R.O.B. directly	Show ready/busy state and virtual-pad-to-game latency.

Acceptance questions

- Capture Test and six Direct commands on the intended LCD/OLED and emulator.
- Demonstrate held unspun press/release of one virtual pad.
- Demonstrate two independent gyro positions and spin-down phases, red/blue button state, and return to holders.
- Measure red/blue-to-A/B mapping, receiver freshness, and actual game gate response. A local visual button state is not proof that the emulator accepted input.

- Test Game A's transmission switch and Game B's direct command cadence.

Original booklet warnings about touching real spinning gyros are historical context. The [virtual system contract](#) is the current product scope.

CHAPTER 04 / CURRENT REFERENCE

Stack-Up manual: design and test notes

Source: [Stack-Up instruction booklet scan](#) at Digital Press. This paraphrases the original accessory and game rules. R.O.B. Vision models the robot and pieces virtually; no physical tray or block is built.

Virtual set and starting layouts

The booklet shows five colored ring-shaped pieces—red, white, blue, yellow, and green—on five numbered trays around R.O.B. Tray 3 is directly in front. Direct and Memory begin with all five on Tray 3, top to bottom red, white, blue, yellow, green. One-player Bingo also begins with five on Tray 3, in any color order. Two-player Bingo begins with three on Tray 3 and one each on Trays 2 and 4. The current UNO Q model initializes the Direct/Memory Tray 3 stack for every Stack-Up session. Bingo-specific starting layouts are historical requirements, not implemented profiles. At the start of a virtual session, the pose is centered over Tray 3, at the highest level, with hands open.

Command and stack behavior

The game sends one of six movement commands—left, right, up, down, open, close—per completed signal. The original robot has five tray stations and six height levels; Stack-Up up/down moves one level, unlike Gyromite's two-level movement. Closing at a lower block can lift that block and every block stacked above it together. The virtual controller therefore carries an ordered **stack segment**, not just one block. Opening places that segment, preserving color order; all five blocks remain accounted for. The game does not send a block ID, destination tray, target pattern, or confirmation that the player's arrangement is correct. Those are inferred from our virtual pose and stack state or seen by the player on the game screen.

The current scripted preview applies one virtual command per step. It first moves red alone from Tray 3 to Tray 4, then closes around blue at level 3 and carries blue plus white to Tray 2. The blocks keep their bottom-to-top order, and all five remain accounted for. Local command buttons use the same virtual stack model. This is not evidence that a game-frame command was received; live play must animate each received primitive independently, including repeated commands and Memory playback.

Modes

Mode	Booklet behavior	Virtual design implication
Test	Optical aiming signal over the ROBOT BLOCK artwork	Press Select to open the mode list, select Test, then Start. The frame-link Test detector recognizes alternating light frames without a virtual movement; live acknowledgement still needs confirmation.
Direct	Professor lands on six command keys	Decode and display left, right, up, down, open, or close. Player scores/advances with Start after arrangement.
Memory	Up to 100 programmed commands at selected speed, then END	Measure cadence; the current model applies commands immediately and has no bounded queue. Report misses during sustained frame-link validation. END is a programming marker, not a robot movement flash.
Bingo, one player	Row or column completion sends a command	Apply only a complete validated optical command.
Bingo, two players	Two human controllers and a different starting layout	Validate separately; keep human controller input on the cabinet.

The booklet says a simultaneous completed row and column is a no-command condition until one line changes. Do not turn that pattern into two virtual actions. The flash stream does not report a target pattern or a victory result. The UNO Q knows its own modeled block stacks but cannot infer the game's scoring outcome without another source. In Direct, Memory, and one-player Bingo, the human presses Start to request scoring after matching the displayed arrangement; a successful animation alone does not advance a round.

Stack-Up's documented modes have no Gyromite-style accessory that presses Controller 2. Its tray model needs no button return path to the game. The browser preview now demonstrates both a one-block transfer and an ordered two-block carry. The live virtual model conserves five blocks and updates a placement on a valid open command. Optical Memory timing still needs hardware validation.

See the [virtual system contract](#), [ROM signal analysis](#), and [verification plan](#).

CHAPTER 05 / CURRENT REFERENCE

Game identification from RetroPie launch events

Status: installed on the dedicated `retropie.local` test machine. Real Gyromite and Stack-Up launches selected the matching game on the UNO Q, and exits cleared it. Controlled start/end calls selected and cleared both games. Launch identification supplies context; the separate game-frame link recognizes movement signals from both ROMs.

Decision

Use the same approach as VirtualGlove: a small RetroPie runcommand launch/end hook sends the game identity. On the dedicated test machine there were no existing custom hooks, so `launch` and `end` scripts were installed. At launch, RetroPie supplies system, emulator, ROM path, and command. Match the **exact ROM basename including .nes, .zip, or .7z**, case-insensitively, against an explicit registry. At exit, clear the active game. On another machine, merge with any existing hook rather than overwriting it.

Only `nes` and `famicom` are eligible. An unregistered filename or other system resolves to idle, not a guessed game. A changed archive filename needs an explicit registry entry. This avoids recognizing an unrelated game merely because its name contains “Gyromite” or “Stack-Up.”

The checked-in registry covers the user's Gyromite (World) and Stack-Up (World) .7z and .zip files plus their named .nes members. A read-only inspection of the supplied archives on 24 September 2026 found one 40,976-byte .nes member per archive. For each game, its .zip and .7z members had identical bytes. No ROM bytes, extracted files, or archive copies were added to R.O.B. Vision.

Resolver and live check

The `resolver` accepts the same four launch arguments used by RetroPie and prints a small JSON event. The `sender` authenticates to the UNO Q using the token file and sends the event with bounded retries. On 24 September 2026, a real EmulationStation Gyromite launch used the .7z archive and selected `gyromite` on the UNO Q. The next real exit cleared the selected game. Controlled calls of the installed hooks also selected and cleared both games.

```
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Gyromite (World).zip" ""
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Stack-Up (World).7z" ""
python3 tools/identify_game.py end
```

The first two calls identify `gyromite` and `stack_up`; the last returns `idle`. The ROM argument is the **launched filename** reported by RetroPie, not the inner `.nes` member of an archive unless RetroPie actually launches that extracted file.

Current cabinet-to-robot path

```
RetroPie runcommand start/end
-> exact local registry match
-> authenticated LAN game-state event
-> UNO Q selects and displays game/accessory context
-> virtual accessory profile and Test checks
-> READY for validated game-frame commands
```

The sender transmits system, ROM path, and launch/exit event over authenticated HTTP with bounded retries. The RetroPie receiver also scans the active RetroArch process and resends identity if the UNO Q restarted during a running game. A launch event is game context only; it does not claim a decoded movement. Browser manual selection is also available while no supported game is active. The game-frame link separately verifies exact ROM identity before accepting a complete command.

Gyromite's paired tray-to-Controller-2 return path remains separate from this launch notification. Stack-Up needs no gyro-style button return in its documented modes. Both games use the Wi-Fi/Ethernet launch event for context and can receive movement commands through the RetroPie frame link.

Remaining validation

1. Simulate all six registered archive/member basenames, an unrelated NES ROM, and another system; verify only intended games match.
2. Confirm extracted `.nes` files through the real RetroPie UI; Gyromite and Stack-Up archives have already been launched and identified.
3. Verify start, exit, unknown game, back-to-back game launches, missed acknowledgement, and sender/robot restart. No stale game may stay armed.
4. Confirm all existing cabinet runcommand actions still execute in their original order and that hook failure never prevents the game from starting.
5. Validate frame-link Test and movement signals for both selected games.

See [network architecture](#), [optical input](#), [configuration](#), and [verification plan](#).

CHAPTER 06 / CURRENT REFERENCE

Optical game input from rendered NES frames

R.O.B. Vision reads the games' light messages from the emulator's rendered frames. There is no camera capture path in the current app. The light protocol still matters: Gyromite and Stack-Up encode left, right, up, down, open, close, and ready-light signals as 13 dark/green frame cells. See the [ROM analysis](#) for the command tables.

Frame classification and validation

The RetroPie wrapper observes each libretro video callback, classifies a grid of NES pixels as dark, green, or other, and forwards one classification and frame number to a local Unix socket. The receiver validates the sending process, wrapper path, and registered ROM. `ExactFrameDecoder` rejects missing frames, non-light frames inside a command, incomplete patterns, and commands belonging to another game. A complete game-specific command travels to the UNO Q's token-protected endpoint. Retries with the same sender PID and frame number apply once.

Test mode is separate from movement. `FrameTestDetector` recognizes a sustained green field in Gyromite or alternating dark/green frames in Stack-Up. A recent authenticated receiver heartbeat makes R.O.B.'s red light blink automatically. The ready-light command makes it steady for at most one second. Neither Test indication moves R.O.B.

Current evidence and limits

Both games delivered all six command types during live FCEUmm Direct play. Nestopia delivered live movement commands from both games. Its Gyromite blue gate return passed Game A checks on Batocera and RetroPie after the Player 2 device fix. The frame path does not depend on display refresh, a camera frame rate, or screen placement. It still requires the R.O.B. Vision emulator launch choice and exact registered ROM identity. Longer sessions and Stack-Up Memory/Bingo remain to be checked. See the [FCEUmm](#) and [Nestopia](#) reports.

Dated Kiyo Pro and live camera reports in this repository document an abandoned research path; their measurements do not describe the current runtime.

CHAPTER 07 / CURRENT REFERENCE

Gyromite and Stack-Up ROM signal analysis

Status: Static analysis of the user-supplied Gyromite (World) and Stack-Up (World) ROMs, 24 September 2026. The ZIP and 7z copies of each game contain identical NES data. No ROM or extracted graphics is stored in this project. The derived patterns are decoded from RetroPie rendered NES frames by the FCEUmm and Nestopia frame-link wrappers.

Run `python3 tools/analyze_rob_roms.py --gyromite '/path/to/Gyromite (World).zip' --stack-up '/path/to/Stack-Up (World).zip'` to reproduce the command report from the archives in place. The program rejects unexpected ROM layout or changed evidence bytes. The inspected full-file SHA-256 hashes are `bf1b323ba39c84b964f93127598b267ff3f16cbf99c533ae1e9b8332232b3e5b` (Gyromite) and `3ab6bc99246783a6bf7083481027fe358ccdb147dcf1165eaf08aaf4e1b06548` (Stack-Up). Each is an iNES mapper-0 image with 32 KiB program ROM and 8 KiB character ROM.

What the games transmit

The screen flashes represent **one robot primitive per command**, not a complete task such as “move the red gyro to its tray” or “solve this Stack-Up pattern.” A 13-frame message has the form:

```
000101w1x1y1z
```

0 is the dark frame (NES palette \$0F), 1 is the green frame (\$2A), and wxyz selects an action. The pattern is consistent with the independently documented NESdev R.O.B. interface and original hardware protocol research. At a roughly 60 Hz NES video rate, the 13 data frames span about 217 ms; the final visible duration and gaps still depend on emulator and display behavior. Stack-Up's transmission routine schedules an additional dark frame after the 13 data frames.

Action	wxyz	Full 13-frame pattern	Gyromite	Stack-Up
Open grippers	1010	00010111101110	Yes	Yes
Close grippers	0110	0001010111110	Yes	Yes

Action	wxyz	Full 13-frame pattern	Gyromite	Stack-Up
Turn left one station	0100	0001010111010	Yes	Yes
Turn right one station	1000	0001011101010	Yes	Yes
Raise two levels	0101	0001010111011	Yes	No
Lower two levels	1101	0001011111011	Yes	No
Raise one level	1100	0001011111010	No gameplay table	Yes
Lower one level	0010	0001010101110	No gameplay table	Yes
Head ready light on	1001	0001011101011	ROM command table	Stack-Up test path

The eight-bit command byte is the message after the five leading bits: for example, open is \$EE, close \$BE, left \$BA, right \$EA, Gyromite up/down \$BB/\$FB, Stack-Up up/down \$FA/\$AE, and ready light \$EB. The ready-light message is a status operation. It must never move an axis.

Gyromite Test produces a sustained green signal. Stack-Up Test alternates green and dark frames on the ROBOT BLOCK artwork. The frame-link Test detector automatically blinks the virtual head light; a separately decoded ready-light command produces a brief steady light. These are status indications, not motion commands.

ROM evidence

- **Gyromite:** The program at \$A679-\$A697 indexes a palette-choice table at \$A6FC, with 13 entries per command slot and the entries read in reverse order. Valid populated slots 1, 2, 3, 5, 6, 7, and 8 decode to the rows above. Slot 4 is deliberately skipped in the controller selection logic at \$A591-\$A5A2; its table area is not a valid optical command. Palette-selection values 3 and 4 resolve through the table at \$B35E to the all-black palette at \$B404 and all-green palette at \$B428 respectively. This ties the table bits to actual on-screen light, rather than just matching known codes by coincidence.
- **Stack-Up:** The routine at \$B1DE starts a transmission and \$B1E7-\$B237 builds successive frames. It uses the template at \$B24A: six fixed preamble bits followed by four variable bits separated by fixed green bits. The variable bits come least-significant-bit first from command values 1-6. Palette selections 2 and 3 point via \$82E8 to the black and green palettes at \$B598/\$B5AC. The ROM's final loop pass is an extra dark frame. A separate test path calls the sender with value 9 for the head light; that is not a movement command.
- **Cross-check:** The byte patterns and action names agree with the [NESdev hardware interface](#). The original [Nintendo photosensing patent](#) describes frame-aligned light code transmission, but this report's per-game command table comes from the supplied ROMs.

These addresses refer to the CPU view of these exact 32 KiB program images (\$8000-\$FFFF). The static analysis establishes the encoded message and palette choices. It does not establish every mode-specific command cadence.

Implemented frame decoder

The FCEUmm and Nestopia wrappers classify each rendered NES frame as dark, green, or other. The local RetroPie receiver uses the verified game identity and complete frame numbers to decode the 13-frame patterns above. It forwards accepted commands to the UNO Q over the paired HTTP link. Incomplete patterns, gaps, unexpected frame classes, and actions outside the selected game's allowed set do not move R.O.B. The ready-light command and Test-mode signal update status only. Separate identical commands remain valid.

Game identification follows the exact launch-filename registry described in [Game Identification](#). It selects the allowed action set and virtual accessory profile; a launch event by itself does not imply a decoded command. Mission reports link activity and accepted or blocked actions.

Implemented virtual action mapping

Every accepted optical message advances **one bounded virtual primitive**. The UNO Q owns the virtual state and the browser animates it. No physical motion controller is part of this design.

Optical event	Robot operation	Display/state requirement
Left / right	Rotate one virtual station	Update station and fixture occupancy; the browser animates the resulting pose.
Gyromite up / down	Move two virtual vertical levels	Move two levels and reject an out-of-range target.
Stack-Up up / down	Move one virtual vertical level	Move one level and update held-block state.
Open / close	Open or close the illustrated grippers once	Update a piece only when the modeled grasp or release is valid.
Ready-light / test	Update optical or visual readiness only	Never move virtual pieces.

For **Gyromite**, a sequence of these primitives transfers a virtual gyro between holder, spinner, red pad, and blue pad. The UNO Q derives virtual pad state from its object model and sends that state to the paired LAN Controller 2 receiver. The flash stream alone does not identify the professor's location or prove that the game accepted the returned button. The model tracks which gyro is held and which pad is pressed.

For **Stack-Up**, the same primitives manipulate five virtual colored blocks across five numbered trays. The pose also includes six height levels; closing at a lower block can carry it and every block above it as an ordered segment. The flash stream has no block ID, target-pattern, or victory message. The UNO Q can know its own modeled stack state, but not the game's desired pattern without another validated source. Memory mode may send rapid commands; the current controller has no queue or explicit missed-command state, so this cadence needs a sustained live frame-link test. In the Bingo simultaneous row-and-column case described in the manual, the game sends no command until one line changes; the frame link should yield no valid movement message.

Next validation gate

Exercise all nine patterns above, idle, Test, Stack-Up Memory speeds, and the Bingo no-command case through both supported wrappers. Compare observed frame classes against the ROM-derived rows and measure missed commands, false actions, and spacing under long sessions.

CHAPTER 08 / CURRENT REFERENCE

Laptop, tablet, and phone dashboard

The running Mission page puts virtual R.O.B. and the accessories first. A browser served by the UNO Q follows its controller snapshot; the same files opened directly through `file://` run the independent scripted preview. Setup shows frame-link and Test status.

Mission layout

The large Live Model animates R.O.B.'s connected shoulders, shared opposing hands, head turn, and vertical carriage. Accessory Bay and Game Table show the same gyro or block state. Gyromite has two holders, a spinner, red/blue pads, and two gyros; Stack-Up has five trays and all five blocks. Pose Preview, Game Table, and System Vitals are grouped beneath the model. The Activity Feed distinguishes manual, optical, and link events. The selected game and RetroPie connection are separate statuses: **GYROMITE / CONNECTED** means Gyromite is selected and the browser reaches the UNO Q. The separate **RETROPIE ONLINE/OFFLINE** indicator reports receiver polling.

Fast Gates appears during live Gyromite. Blue (2), red (1), and Release Both (0) act immediately; each hold expires after 60 seconds. A manual pose preview or scripted demo is not evidence of a decoded game command. Demo mode can run while RetroPie is paired and idle; a game launch stops it.





Setup layout

Setup contains RetroPie pairing and **Check Link**, frame-link status, Test-mode frame watching and red-light preview, Gyromite pad checks, and Stack-Up movement checks.

State and limits

dashboard/live.js polls /api/state every 500 ms and enters live mode when the controller responds. The UNO Q owns game, pose, pieces, and pad state; a browser reload simply resumes from the latest snapshot. The last 30 controller events are display history, not a guaranteed command log. The browser cannot verify Hector's location, Stack-Up scoring, or a visible gate response.

The static preview demonstrates a finite Gyromite spin/return sequence, Stack-Up block transfers, and Pose Lab. Its graphics are original SVG/CSS art informed by the [historical manual notes](#). Frame-driven animation was observed in both supported games.

CHAPTER 09 / CURRENT REFERENCE

R.O.B. Vision network architecture

The UNO Q owns virtual R.O.B. state. A browser on the trusted LAN renders it, and RetroPie supplies game identity and receives Gyromite pad states. RetroPie sends complete rendered-frame commands to the UNO Q. No USB game-data tether is used.

```
RetroPie runcommand ■■ authenticated launch/exit HTTP ■■> UNO Q
RetroPie receiver <■■ authenticated /api/state polling ■■ UNO Q virtual pads
Browser <■■ /api/state every 500 ms ■■ UNO Q controller
FCEUmm/Nestopia frames ■■ local verified socket ■■> RetroPie receiver
RetroPie receiver ■■ authenticated command HTTP ■■> UNO Q model
```

The App Lab gateway serves `http://arduiain.local` on port 80 and forwards to the controller on 8766. Existing Avahi supplies the `.local` name. Wi-Fi or an Ethernet adapter carries the same HTTP traffic. The Ethernet adapter shares the current USB hub.

Identity and safety

Exact configured RetroPie ROM basenames select `gyromite` or `stack_up`; unknown titles clear context. The launch notifier, receiver poll, and game-frame command POST use the shared token. Pairing sends that token to RetroPie after checking a short-lived code and TLS certificate fingerprint. Browser controls intentionally require no token on the trusted LAN, so neither port should be exposed publicly.

The receiver is a root uinput service that polls about every 50 ms. It releases both buttons after 750 ms without a good response, on game exit, or when no matching RetroArch process is active. It scans the running process and can resynchronize game identity after a UNO Q restart. Its authenticated poll also supplies the online indicator, which expires after three seconds.

The UNO Q's `/api/state` response is a schema-versioned snapshot with recent events. There is no push stream, durable event log, sequence protocol, or receiver acknowledgement of an on-screen gate animation. Browser refresh gets the current state. The RetroPie frame link reports its fresh activity in `/api/state.input.frame_hook`.

CHAPTER 10 / CURRENT REFERENCE

R.O.B. Vision UNO Q hardware setup

R.O.B. and every accessory are virtual. The physical build consists of an Arduino UNO Q, a RetroPie game host, network connection, and a browser display. No robot mechanism, accessory sensor, or camera is needed.

Power the UNO Q with a suitable supply and keep its vents clear. The tested setup uses Ethernet through a USB hub; Wi-Fi carries the same traffic. Open `http://arduinoai.local/dashboard/` on a laptop, iPad, or phone. The UNO Q's 13×8 LED matrix provides small status animations.

On RetroPie, install the launch hooks, paired Controller 2 receiver, and R.O.B. Vision FCEUmm or Nestopia frame-link choice. Start either registered game and verify **GAME FRAMES LINKED** on Mission. Both Gyromite gate colors were confirmed in Game A. Check Test mode and each movement command with the [verification plan](#).

See [installation](#) for deployment steps.

CHAPTER 11 / CURRENT REFERENCE

R.O.B. Vision parts plan

The build is an electronic controller with a virtual R.O.B. display. Historical gyros, spinner, pads, trays, arms, and blocks are software models, not parts to buy.

Item	Current status
Arduino UNO Q	In use for App Lab controller, HTTP panel, virtual state, and LED matrix sketch.
RetroPie game host	<code>retropie.local</code> has launch hooks and a paired virtual Controller 2; both Gyromite gate colors have responded.
UNO Q power and enclosure	Use adequate supply and ventilation; an enclosure is optional.
Wi-Fi or Ethernet adapter	Current setup uses an Ethernet adapter on a USB hub; Wi-Fi uses the same app protocol.

Item	Current status
Laptop, tablet, or phone	Main browser display for R.O.B. and game accessories.

No servo, motor driver, home switch, physical gyro, tray sensor, spinner motor, or actuator supply belongs to this project.

CHAPTER 12 / CURRENT REFERENCE

R.O.B. Vision configuration reference

Status: Current code defaults and installed test-machine behavior as of 25 September 2026. R.O.B. Vision uses console game frames for automatic input; it has no camera dependency.

Setting	Current value or location	Meaning
App Lab gateway	HTTP port 80	http://arduia.in.local/dashboard/ forwards to the controller.
Controller service	Port 8766	Direct /dashboard/ and /api/ access. Local CLI binds loopback by default; use --host 0.0.0.0 for a trusted LAN.
Controller token	UNO Q app data/controller-token, overridden by ROB_VISION_TOKEN when set	The installer creates and preserves the private file. It authenticates RetroPie launch events, game-frame commands, and receiver polls. Browser controls on the trusted LAN do not use it.
Gyro spin lifetime	55 seconds	Illustrative virtual lifetime.
Fast Gate hold	60 seconds maximum	Each manual Gyromite press automatically releases.
Browser state polling	500 ms	/api/state snapshot; recent events capped at 30.
RetroPie receiver polling	50 ms; request timeout 250 ms	Root uinput service reads the UNO Q state.

Setting	Current value or location	Meaning
RetroPie stale release	750 ms since last good response	Both virtual buttons release; game exit and missing RetroArch also release them.
RetroPie process scan	250 ms	Identifies an active RetroArch launch and can resync UNO Q game context, throttled to two seconds.
RetroPie game-frame socket	<code>/run/rob-vision/frames.sock</code>	The FCEUmm or Nestopia proxy sends one classified cell per rendered NES frame. The receiver verifies sender credentials, approved proxy path, ROM identity, and complete 13-frame commands.
RetroPie per-game NES choice	<code>lr-robvision-fceumm</code> or <code>lr-robvision-nestopia</code>	Both use the installed original core through the frame wrapper. Current Gyromite and Stack-Up selections remain FCEUmm. Plain <code>lr-fceumm</code> and <code>lr-nestopia</code> bypass the link. For a per-game core switch after installation, the low-level <code>tools/install_retropie_frame_hook.py</code> accepts <code>--gyromite-core</code> and <code>--stack-up-core</code> .
Game-frame indicator	Last matching receiver heartbeat within one second	<code>/api/state.input.frame_hook</code> is true.
Test signal indicator	Sustained green or alternating rendered frames	The receiver reports a recent signal automatically; the UNO Q blinks R.O.B.'s red light.
Receiver online indicator	Authenticated poll within three seconds	Otherwise Setup and Mission show RetroPie offline.
Game registry	<code>config/games.json</code>	Exact case-insensitive ROM basenames for Gyromite and Stack-Up, including configured ZIP, 7z, and NES names.

Release installation

The versioned installer URL and supported hardware are listed in the [installation guide](#). The Uno Q uses `uno-q`; both consoles use `console <Uno-Q-hostname>`. The same command upgrades an existing installation. Add `--pair` to the console command after revoking its credential in Setup. Batocera 43.1 x86_64 is the validated Batocera build; other board/version combinations are rejected before installation.

Local commands

```
python3 -m controller.service --host 127.0.0.1 --port 8766
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Gyromite (World).zip" ""
python3 tools/identify_game.py end
```

The optional standalone `deploy/rob-vision.service` must stay disabled while App Lab owns port 8766. App Lab runs one app at a time on this UNO Q. The repository includes UNO Q shutdown, early-start, and network-status host units; App Lab does not install them. Existing Avahi provides `arduinain.local`. See [UNO Q host helpers](#).

CHAPTER 13 / CURRENT REFERENCE

R.O.B. Vision safety and security

R.O.B. and every accessory are virtual. There are no motors or physical gyros. The relevant risks are incorrect game input, stale network state, untrusted browser access.

- Mission and Setup browser controls intentionally work without a token on the trusted LAN. Keep ports 80 and 8766 away from guest and public networks. Browser writes use same-origin JSON checks; this is not a substitute for network isolation.
- RetroPie launch notices, receiver identity, and emulator-frame commands require the shared controller token. Pairing uses a short-lived code and checked TLS certificate fingerprint. Do not publish the token or include it in logs.
- The optical decoder rejects incomplete and ambiguous flashes. The RetroPie frame link also requires a complete 13-frame pattern, matching ROM identity, and verified local sender credentials. Missing or malformed frame data does not repeat the last action.
- Virtual movement validates bounds and block conservation. Invalid Stack-Up transfers do not move pieces.
- The RetroPie receiver releases both Gyromite buttons on exit, unknown game, missing process, or 750 ms without a good response. Fast Gate holds expire after 60 seconds.
- Live **Emergency Stop** clears game selection and releases virtual pads. In the offline preview it only cancels the local animation. It is not a physical power switch.
- User ROMs are not bundled.

The original booklet's warnings about a physical spinning gyro are historical context for Nintendo's toy, not a hazard from this virtual implementation.

CHAPTER 14 / CURRENT REFERENCE

R.O.B. Vision verification plan

Current evidence: the preview, UNO Q app, real game launch and exit identification, live red/blue Gyromite gate response, and independent model flows have been verified. The RetroPie game-frame link decoded all six commands in both games during live Direct play and drove the UNO Q model. See the [game-frame report, unattended tests](#).

Gate	Test	Pass condition	Status
Gyromite preview	Run the unspun one-gate press, dual relay, spin-down recovery, and cleanup	Pad states stay independent; after one recovery per gyro, both gyros return upright to holders, buttons release, and COMPLETE remains stable	Browser preview observed.
Preview reset	Use Home and demo restart at different phases	Timers, queue, pieces, and buttons return to initial state	Browser implementation; regression tests pass.
Stack-Up preview	Move red alone, then blue and white together; try a blocked release	Five blocks conserved, colors stay ordered, and invalid moves leave state unchanged	Browser observed; five model tests pass.
Frame Test signal	Enter Test mode in both games while the frame link is active	Red Test light blinks while no movement occurs	Local automated tests pass; live frame-link Test check remains to be confirmed.
RetroPie game-frame link	Launch each registered ROM through EmulationStation and issue six Direct commands	Exact 13-frame command recognized, authenticated receiver forwards it once, UNO Q model applies or correctly blocks it	All six command types observed in both games; model state changed for valid commands and blocked invalid boundaries. Longer session and Memory/Bingo coverage remain. See the frame-link report .
Game identity	Launch exact configured Gyromite/Stack-Up names and unknown titles	Correct context on start; idle on exit/unknown	Real Gyromite . 7z launch and exit selected then cleared UNO Q context. The receiver now recovers Gyromite context after a UNO Q restart during a live game. Both games' installed hooks passed controlled start/end tests; Stack-Up booted headlessly in RetroArch. A real interactive Stack-Up . 7z launch selected Stack-Up and its exit cleared the UNO Q. An unknown-title launch remains pending.

Gate	Test	Pass condition	Status
Gyromite LAN output	Send red/blue virtual states and disconnect	Correct Controller 2 mapping; button releases on virtual lift, timeout, exit, and reset	Both colors pressed/released at Linux input with Player 2 forced to Gamepad. Initial isolated red moved the blue gate. After swapping A/B and restarting, isolated blue moved the blue gate and released it. The player later confirmed both red and blue controls work in Game A. Automated stale-release checks pass; sustained physical link-loss observation remains pending.
Multi-client UI	Open laptop, iPad, and phone simultaneously	Same UNO Q snapshot and ordered events; reconnection does not reset game	Not run.

CHAPTER 15 / CURRENT REFERENCE

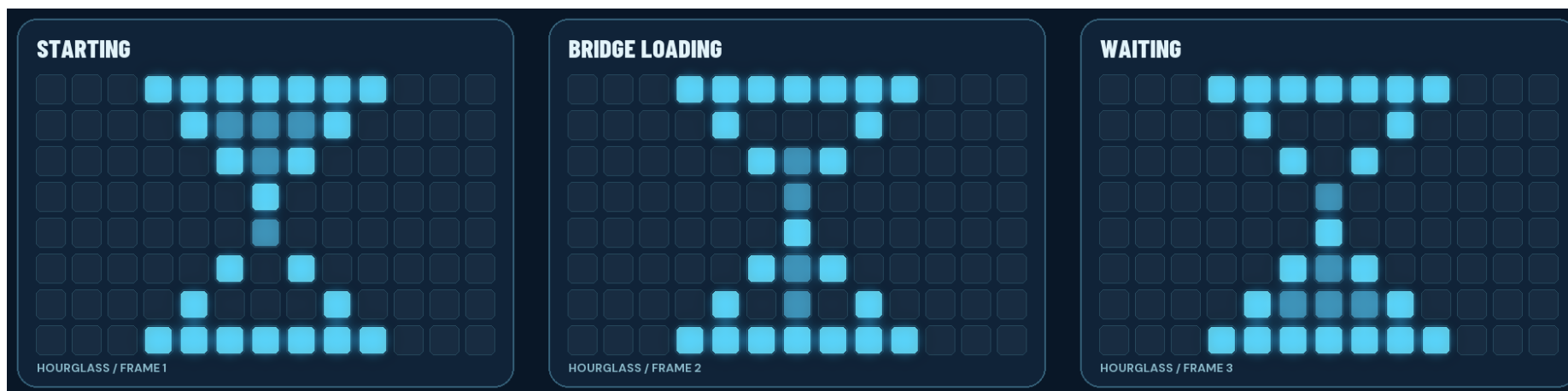
UNO Q matrix display design

Status: implemented in the R.O.B. Vision App Lab sketch and Linux bridge; physical display behavior still needs direct visual confirmation. RetroPie pairing code is generated on RetroPie and entered on the browser Setup page. The UNO Q matrix shows a pairing cue only. This guide defines the behavior for the built-in 13-column by 8-row blue LED matrix. The large browser dashboard remains the place for the full R.O.B. animation, accessories, and explanations.

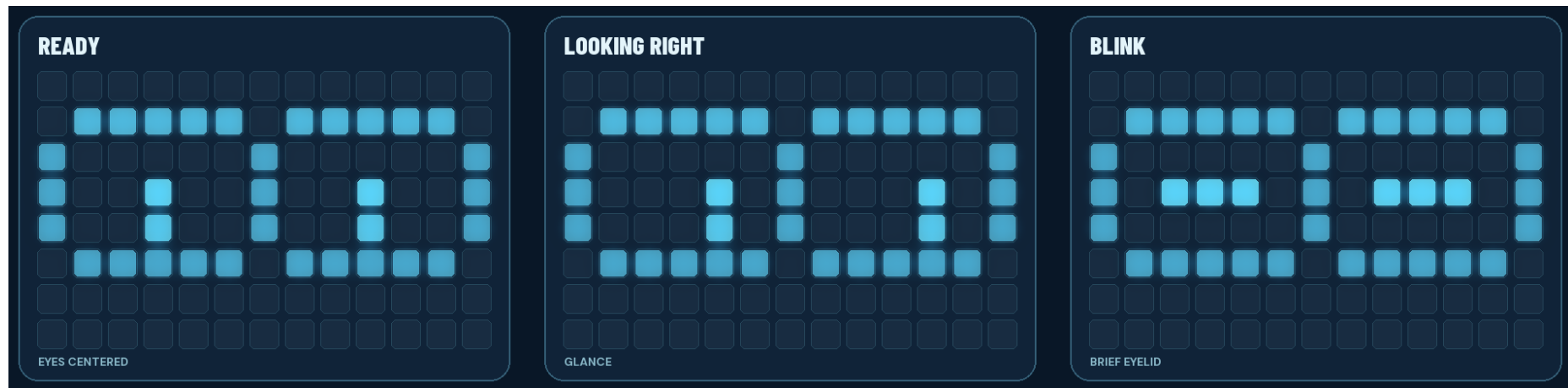
In Buddy's story, this tiny display is his first sign of life outside the emulator. These images show the actual 13×8 display states defined in `sketch/sketch.ino`. Each panel is one frame of an animation; Buddy's full-color guide portrait is an illustration, not a matrix state.

The animations

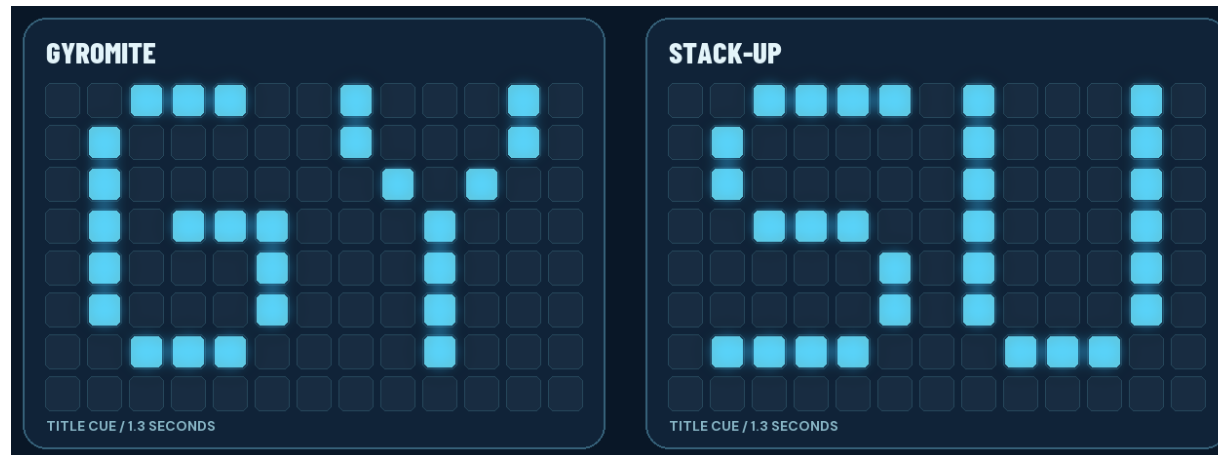
Starting and reconnecting. Three hourglass frames repeat until the controller is ready.

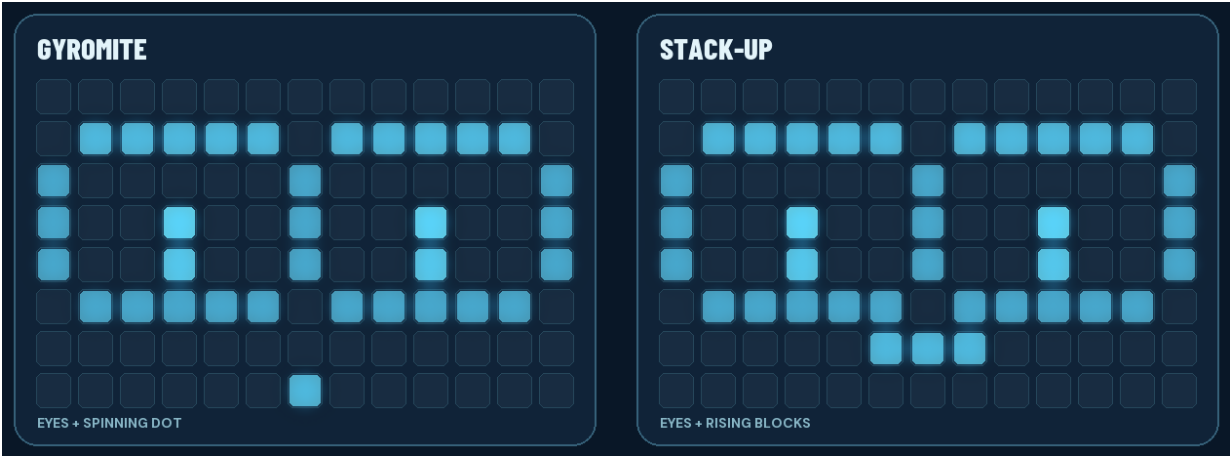


Ready, glance, blink. The eyes move slowly while R.O.B. waits for a game.

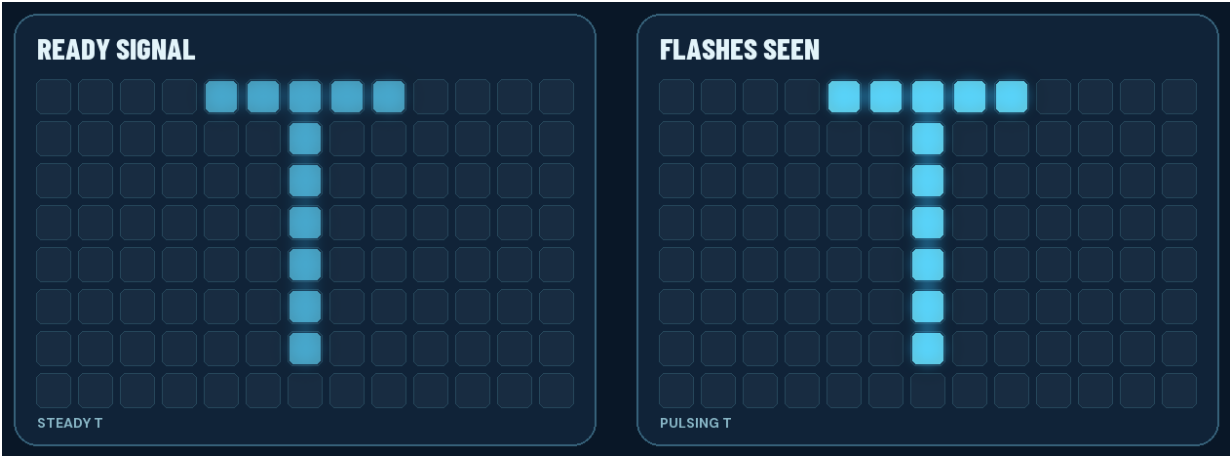


Game selected. A brief title cue identifies the game; the eyes then return with a small decorative accent. The accents do not indicate a physical gyro or block movement.



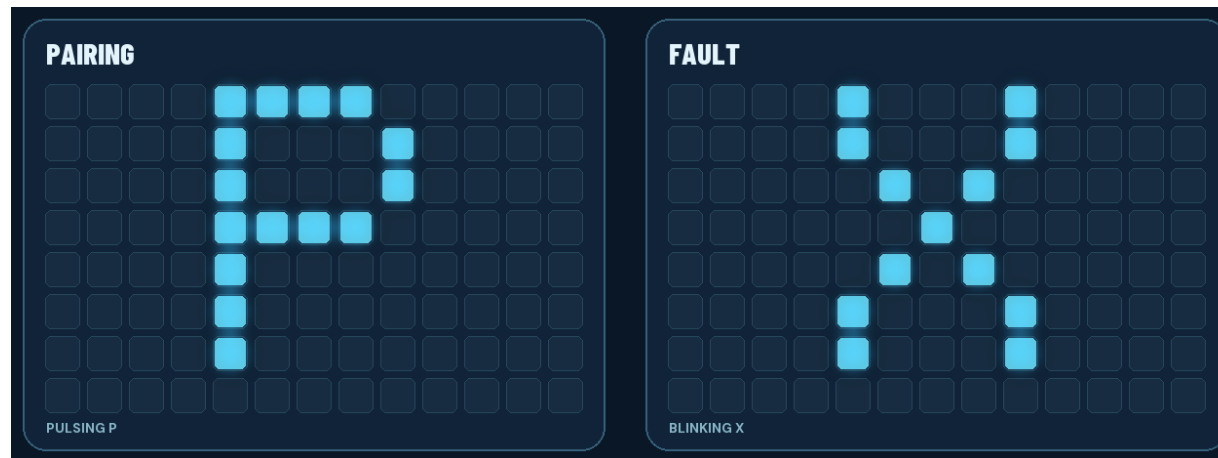


Game Test and faults. The T gently pulses while a game Test signal is detected and stays steady for up to one second after a separate ready-light command. It then returns to the game eyes. P pulses during pairing; X blinks for a fault.



What people should see

State	Matrix	Meaning
Board power-on, before our sketch runs	Arduino's own boot graphics	System startup; R.O.B. Vision cannot own the matrix yet.
R.O.B. Vision sketch starts	VirtualGlove-style pulsing hourglass	The app and Linux side are starting. Begin it as early as the platform permits, before waiting for Router Bridge setup.
Linux bridge connects but the controller is still starting or reconnecting	The same hourglass, restarted or continuing smoothly	The bridge is available; the controller has not yet reported ready. A bridge connection alone must not imply game readiness.
Controller ready, no game selected	A pair of curious R.O.B. eyes: slow left/right glance and occasional blink	R.O.B. is awake and waiting for a game. This is an attract animation, not proof that a game is linked.
Gyromite selected	Brief GY, then eyes with a small spinning-dot accent	The selected game is Gyromite. The accent is decorative; it must not claim a gyro is actually spinning.
Stack-Up selected	Brief SU, then eyes with a small rising-block accent	The selected game is Stack-Up. The accent is decorative; it must not claim a block was moved.
Valid movement command accepted	Eyes glance in the movement direction, rise/fall for vertical motion, or narrow briefly for grip	Mirror the authoritative virtual action, then return to the selected game's eye loop. Never animate from an undecoded flash alone.
Test flashes detected	T gently pulses	The frame link has reported the game's Test signal automatically.
Ready-light command detected	Brief steady T	The game sent a separate ready-light command; the game eyes return within one second.
Pairing in progress	Large P with a slow pulse	Open the browser Setup page and read the pairing code and fingerprint shown on RetroPie. Do not show invented or unrelated digits on the UNO Q.
Controller fault	Blinking X	Open Setup for the actual problem; the matrix alone cannot explain it.
App stopped	Matrix released/blank as the platform permits	R.O.B. Vision is no longer driving the display.



The face should be drawn for the physical 13×8 grid, with two distinct eyes and enough dark space to make a blink or sideways glance readable. The blue LEDs are monochrome: eye shapes and brightness, not color, carry the expression. Limit idle motion to an occasional glance or blink so it feels alive without competing with the game. Show GY/SU long enough to identify the title, then let the eyes take over. A brief repeat of the title mark after a long idle period is acceptable.

State ownership and priority

The sketch owns all framebuffer writes and animates the startup hourglass without waiting for Linux. Linux sends compact requested states and accepted action hints through Router Bridge, using the UNO Q controller's authoritative game, frame-link, test, and fault state. It sends a heartbeat at least once per second. If Linux disappears for about 3.5 seconds, the display returns to the hourglass rather than freezing a misleading game face. The sketch cannot replace Arduino's protected boot logo before App Lab releases the microcontroller.

Display priority: **fault** → **pairing** → **active Test or ready signal** → **startup/reconnect** → **selected game** → **idle eyes**. A movement hint is temporary within the selected game and never overrides fault, pairing, or Test. Frame-link status is visible on Setup; the idle eyes alone do not claim a linked game. Pairing has a bounded lifetime and returns to the prior state when complete or expired.

Pairing distinction

VirtualGlove has a separate physical pairing display for its own device approval PIN. R.O.B. Vision pairs **RetroPie to the UNO Q**: RetroPie produces a six-digit code and certificate fingerprint, while the UNO Q browser Setup page accepts them. The matrix P is a status cue when **Pair another console** is opened or pairing is submitted; it expires or clears afterward. Showing the code on the UNO Q would require a deliberate change to the pairing protocol.

Physical acceptance checks still needed

1. Cold boot shows the protected system graphics, then the hourglass as soon as the R.O.B. Vision sketch can run; the hourglass remains through Router Bridge initialization.
2. When the Linux controller is ready, the hourglass changes to the idle eyes. Stopping/restarting Linux restores the hourglass and then the correct current game display.
3. Real Gyromite and Stack-Up launch/exit notifications select and clear their distinct marks and eye loops.
4. Test T, pairing P, and fault X follow the priority and timeout rules. No matrix state claims a Test signal or game action that the controller has not observed.
5. Verify brightness and animation are comfortable beside the game display.